MASTER PROMPT: "Design Code Bridge" (Figma plugin + local MCP server)

This is one complete prompt. It has six parts. Read all of them before writing any code.

- Part 1: Product and file-by-file specification
- Part 2: Architectural patterns and the invariant each must protect
- Part 3: Dependencies, package.json and module wiring
- Part 4: Required function inventory per file
- Part 5: Requirements for complex designs
- Part 6: Delivery, tests and acceptance

Ground rules (apply to everything below)

1. Deliver every file in full. No placeholders, no "...", no "rest unchanged".
2. Use only the dependencies in Part 3. CommonJS only. No OCR, no image-resize, no extra libraries.
3. Where a number is given, use that number. Where something is unspecified, decide and record the decision in DESIGN.md.
4. Never produce an image where real code is possible. Text is never an image.
5. Never silently drop content. If something is truncated, approximated or skipped, report it.
6. Deliver one file per message and wait for "next" if the output is long. Do not shorten files to fit.

PART 1: PRODUCT AND FILE-BY-FILE SPECIFICATION

0. What it is

A Figma desktop plugin plus a local Node.js server that gives an AI coding agent (GitHub Copilot Agent mode, or any MCP client) an exact, structured, language-neutral spec of the current Figma selection, and instructs the agent to write real code in the stack the user names. Output is code, never pictures.

```
Figma plugin UI (iframe) <--ws://localhost:PORT/figma--> server.js <--
http://localhost:PORT/mcp (MCP, streamable HTTP)--> agent in any VS Code window
Figma plugin main thread <--postMessage--> plugin UI iframe
```

Default PORT 3845, overridable by env PORT. Server binds 127.0.0.1 only. Design data is pulled from Figma on every tool call and never cached.

1. Files to deliver

manifest.json, code.js (plugin main thread), index.html (plugin UI), server.js, guard.js, targets.js, reference.js, analyze.js, verify.js, package.json, README.md, copilot-instructions.example.md, DESIGN.md, tests/.

package.json: name figma-code-bridge, version 3.0.0, node >=18, CommonJS. Dependencies: @modelcontextprotocol/sdk ^1.17, ws ^8, zod ^3.25, pngjs ^7, jpeg-js ^0.4, imagetracerjs ^1.2 (optional at runtime: tracing is skipped if missing). Scripts: start (node server.js), seal (node guard.js seal), verify (node guard.js verify), unseal (node guard.js unseal), check-output (node verify.js). bin entries for server.js and verify.js.

manifest.json: Figma plugin manifest, name "Design Code Bridge", main code.js, ui index.html, editorType figma, documentAccess dynamic-page, networkAccess allowing only http://localhost:PORT and ws://localhost:PORT. Use a current Figma API version; verify field names against the Figma plugin docs.

2. Plugin main thread (code.js)

Style constraints: ES5-style `var`, no optional chaining or nullish coalescing (sandbox safety). `figma.showUI(__html__, {width:380, height:520, themeColors:true})`.

Messaging: on UI message `ui-ready`, post `{type:'bridge-ready'}` then announce selection. On `selectionchange`, post `{type:'selection', pageName, nodes:[{id,name,type,width,height}]}`. On `{type:'request', id, tool, args}`, run `handlers[tool](args)` and reply `{type:'response', payload:{id, ok:true, result}}` or `{ok:false, error}`.

Constants: ICON_MAX 64, MAX_EXPORT_FILES 150, SVG_INLINE_MAX 12000 chars per inlined SVG, SVG_TOTAL_BUDGET 120000 chars per spec, default maxNodes 600, default depth 12.
Type groups: VECTOR_TYPES = VECTOR, BOOLEAN_OPERATION, STAR, POLYGON. SHAPE_TYPES = those plus LINE, ELLIPSE, RECTANGLE. CONTAINER_TYPES = FRAME, GROUP, INSTANCE, COMPONENT.

2.1 Handlers (tools the plugin implements)

- get_selection({depth,maxNodes,id}): spec of selected roots (or id).
- get_node({id,...}): spec of one node.
- convert_selection({id,depth,maxNodes,exportAssets,svgFiles}): returns `{spec, files, failed, skipped, complete}`; files come from exportAll unless exportAssets===false.
- export_assets({id,svgFiles}): uses maxNodes 2000, depth 30; returns exportAll result plus specComplete, specErrors, specTruncated; complete = result.complete && spec.stats.complete.
- get_svg({id}): `exportAsync({format:'SVG_STRING', svgIdAttribute:false, svgOutlineText:true, svgSimplifyStroke:true})` -> {id,name,svg}.

- get_page_outline(): current page top-level visible nodes {id,name,type,w,h}; SECTION nodes include children; plus selection ids.
- get_screenshot({id,scale}): PNG, default scale 1, capped so the longest side <= 2048px; returns base64.
- get_tokens(): all local variable collections (per mode values; aliases rendered as {alias:name}; colors via toColor), local text styles (family, style, size, lineHeight, letterSpacing), local paint styles (first visible solid color).
  Selection resolution: if no id and nothing selected, throw "Nothing is selected in Figma. Select a frame or component first." Ids of DOCUMENT/PAGE are rejected.

2.2 Node serializer (the spec)

Skip invisible nodes. Each node yields: id, name, type, w, h (rounded to 2 decimals), x/y only when the parent is not auto-layout (or the child is absolutely positioned inside auto layout, then `positioning:'absolute-in-auto-layout'`). For children of GROUP/BOOLEAN_OPERATION, express x/y relative to the group (compute from absoluteTransform), and mark `group:true` for GROUP (pure wrapper, no styling). Also: constraints {h,v} when not MIN/MIN for children of non-auto parents; rotation; opacity (if != 1); blendMode (if not NORMAL/PASS_THROUGH); mask:true; sizing {h,v} (layoutSizingHorizontal/Vertical) for auto-layout nodes and their children; grow, selfAlign (if not INHERIT); min/max width/height; clip when clipsContent.
Layout: auto-layout containers -> `layout:{direction (HORIZONTAL|VERTICAL|GRID), gap, padding{top,right,bottom,left}, mainAlign, crossAlign, wrap?, crossGap?}` and for GRID rows/columns/rowGap/columnGap. Non-auto containers with children -> `layout:{mode:'absolute'}` (later siblings paint on top).
Instances: component (main component name via getMainComponentAsync), componentSet, props (component properties with the "#id" suffix stripped); count instances per component name for a top-level `components` list.
Fills/strokes: solid {type:'solid', color, variable?}; gradients {type:'linear|radial|conic|diamond-gradient', stops:[{at,color}], handles, transform, cssAngle (linear only)}; image {type:'image', scaleMode, assetName:'img-'+first 10 chars of hash, transform, scalingFactor, rotation, filters, opacity}. Colors: `#rrggbb` when alpha >= 0.999 else `rgba(r, g, b, a)`. cssAngle: invert the gradient transform, take points (0,0.5) and (1,0.5), scale by node w/h, angle = atan2(dx, -dy) in degrees normalized to 0-359.
Strokes: strokes[], strokeWeight (number or per-side), strokeAlign, dash. Also fillStyle name, radius (number or per-corner object), effects (drop-shadow/inner-shadow {x,y,blur,spread,color}, layer-blur, background-blur), `vars` (bound variable names per property; also fill the global tokens map name->value).
Text: use getStyledTextSegments (fontSize, fontName, fontWeight, lineHeight, letterSpacing, fills, textDecoration, textCase, textStyleId, hyperlink, listOptions). `text:{characters (U+2028 -> \n), align, valign, autoResize, truncation/maxLines, paragraphSpacing, style (if one segment) | segments:[{text,style}]}`. Style: fontFamily, fontStyle, fontWeight, fontSize, lineHeight ('auto' | 'Npx' | 'N%'), letterSpacing ('Npx' | 'Nem' (percent/100)), color/colorVar, decoration, textCase, textStyle, link, list. Collect up to 400 non-empty text strings into top-level `texts`. Collect font "family | style" set into `fonts`.

Limits: if node count reaches maxNodes, emit stub `{id,name,type,truncated:true}` and set truncated. If depth reaches maxDepth, emit childCount + truncatedChildren[{id,name,type}] + truncated:true. Per-node exceptions are caught into node.error and spec.errors[{id,name,message}] (never abort the whole spec).
Top-level spec: {file, page, notes[] (explain units, x/y rules, sizing semantics, group wrappers, pure-code asset rules, image-placeholder, hint), nodes, components, fonts, tokens, assets[{kind,assetName,name,reason,nodeId,inline}], errors, texts, summary{distinctColors, solidFills, grayRatio, gradients, imageFills, effectNodes, placeholders, fontFamilies}, stats{nodes, truncated, complete = !truncated && no errors}}. grayRatio = share of solid fills with HSV-style saturation < 0.12.

2.3 Semantic hints and placeholders

Hint = first match of name against case-insensitive regexes, in this order: button (button|btn|cta as a word), input (input, text field, search, dropdown, select, combobox, textarea), checkbox, radio, toggle (toggle|switch), tab (word "tab(s)"), nav (navbar, nav bar, navigation, menu, sidebar, top bar, app bar), header, footer, card (words), list-item (list item | row | cell as words), badge (badge, chip, tag, pill), avatar (avatar, profile pic), divider (divider, separator), modal (modal, dialog, popup, bottom sheet). Additionally: LINE nodes, and RECTANGLEs with min(w,h) <= 2, get hint `divider`.
Placeholder (`role:'image-placeholder'`) applies to container/RECTANGLE/ELLIPSE nodes with no image and no text in the subtree when either (a) "X-box": has 1-3 visible children, w and h >= 24, each child is a LINE rotated 5-85 or 95-175 degrees (mod 180) with width >= 0.7*diagonal, or a VECTOR covering >= 70% of the box in both dimensions; or (b) name matches (image|img|photo|picture|placeholder|thumbnail|thumb|cover|banner|hero|avatar|media|video) as a word, w,h >= 16, and it is a RECTANGLE/ELLIPSE or an empty container. Placeholders are not descended into and never become assets.

2.4 Asset decision (what becomes SVG/PNG)

For container nodes (and VECTOR_TYPES always -> svg, reason vector), in this priority order:

1. Has a mask child and no text in subtree -> png scale 2 if image fills inside else svg (masked-group).
2. Name (+component name) matches logo|logotype|logomark|wordmark|emblem|illustration|artwork, text count <= 3 and has vectors or images -> svg/png scale 2 (logo-or-illustration).
3. Not root, has exportSettings, no text, has vectors/images -> format from first export setting (SVG, or PNG/JPG with its scale) (export-settings).
4. Vectors present, no images, w,h <= 64, only shapes inside -> svg (icon).
5. Not root, not auto-layout, no text, >= 3 vectors, no images, only shapes -> svg (vector-cluster).
   Subtree stats (text/vector/image counts of visible descendants) are memoized per node. For svg assets, export SVG_STRING (options as get_svg); inline it as `asset.svg` if length <= 12000 and the running total <= 120000, else set `asset.svgRef:true`. The node's asset descriptor: {id, kind, assetName: slug(name)+'-'+id with non-alphanumerics replaced by _, reason}. An asset node is a leaf (no fills/children serialized). Image fills register an asset {kind:'image', assetName, hash} once.

2.5 Asset export (exportAll)

Skip svg assets that were inlined unless svgFiles is true. Image assets: figma.getImageByHash -> bytes, sniff type by magic bytes (png/jpg/gif/webp/bin), base64. Node svg -> utf8 SVG file. Node png -> PNG at asset.scale (default 2) base64. Cap at 150 files; return {files: [{assetName,fileName,mimeType,encoding,data}], failed:[{assetName,error}], skipped:[assetNames beyond cap], complete: no failed and no skipped}.

3. Plugin UI (index.html)

Small themed dashboard (light/dark, uses Figma theme vars): header with connection state ("Connected to bridge" / "Not connected"), "Current selection" card (list of selected nodes with name/type/size and an updated timestamp, or "Waiting for Figma selection…"), "Recent activity" card (latest 6 events with time), footer "Keep this panel open to sync your Figma selection with Copilot." Behavior: open WebSocket to ws://localhost:PORT/figma; auto-reconnect with a retry delay; on open post `{pluginMessage:{type:'ui-ready'}}` to the parent. Relay: incoming WS `{id,tool,args}` -> `parent.postMessage({pluginMessage:{type:'request',id,tool,args}},'*')`. Messages from the plugin: `selection` -> update UI and forward `{type:'selection',pageName,nodes}` to the server over WS; `response` -> send `payload` over WS as `{id,ok,result|error}`. If not running inside Figma, show a hint to open it via Plugins > Development. Never use innerHTML with untrusted names (use textContent).

4. Server (server.js)

4.1 Transport and plugin link

- HTTP: GET /health -> {ok:true, pluginConnected, integrity:status}. POST /mcp -> MCP streamable HTTP, stateless (new McpServer + StreamableHTTPServerTransport with sessionIdGenerator undefined per request; close both when the response closes). Any non-POST on /mcp -> 405 JSON-RPC error with Allow: POST and a message pointing to /health. Anything else -> 404. Reject /mcp requests whose Host header is not localhost/127.0.0.1/[::1] or that carry an Origin header (403) (DNS-rebinding defence).
- WebSocket upgrade only on /figma, with valid Host and Origin absent or the literal "null" (Figma iframes); else write 403 and destroy. ws maxPayload 200 MB. Only one plugin connection: a new one closes the previous. Log selection changes (page + up to 8 node names/types/ids, "+N more") and connect/disconnect.
- callPlugin(tool,args): rejects with a friendly "plugin not connected, open Plugins > Development > Design Code Bridge and keep the window open" if no open socket; sends {id: uuid, tool, args}; 60 s timeout ("Timed out waiting for Figma. Try a smaller selection, a lower depth, or exportAssets:false."); on socket close reject all pending.
- Lifecycle: exit on SIGINT/SIGTERM and when the launching parent process (process.ppid) disappears (poll every 1 s with process.kill(pid,0), unref'd timer); close sockets, close HTTP, force exit

after 2 s. Friendly EADDRINUSE message. Startup log lists MCP URL, WS URL, bridge dir, allowed roots, integrity warning or "run npm run seal" tip.

4.2 Safety (guard.js + server)

- guard.js CLI: `seal` hashes (sha256) a fixed list of bridge files into `.bridge-seal.json` ({sealedAt, files}) and chmods them (and the seal file) 0444; `unseal` chmods 0644 and deletes the seal; `verify` prints check() JSON and exits 1 unless status ok. check() returns status unsealed | ok | MODIFIED with changed[] (missing or unreadable files count as changed). Exports BRIDGE_DIR (realpath of its folder), FILES, seal, unseal, check, assertSafeOutputDir, assertSafeSubdir.
- assertSafeOutputDir: must be an absolute path and must not be the bridge folder or inside it. assertSafeSubdir: relative, no `..`.
- Server resolveOutputDir: guard check on the given path, then again on the nearest existing real path (to defeat symlinks); if env FIGMA_BRIDGE_ALLOWED_ROOTS is set (path.delimiter-separated), the output dir must be inside one of those real roots and no path segment below the root may be a symlink.
- Every JSON tool result includes `bridge:{rules, integrity?}`. rules text: the bridge folder is read-only infrastructure, never edit it, report problems in chat, output must be pure code, call verify_conversion when done. If integrity is MODIFIED, add integrity{status, changed, warning:"STOP and tell the user; do not edit the bridge to fix anything"}.
- Asset writes never overwrite: use flag 'wx'; on EEXIST compare bytes: identical -> counts as written (unchanged:true), different -> failed entry. File names are sanitized (basename, [a-zA-Z0-9._-]).

4.3 MCP tools (register with zod input schemas, descriptions written for an LLM)

1. **convert_selection** (primary). Inputs: framework (string, required), outputDir (absolute, required), styling, language, notes, fidelity (auto|lofi|hifi), name, id, assetsSubdir (default assets/figma), exportAssets (default true), traceRasterLogos (default true), svgFiles (default false), reference (inline|file|none, default inline), depth (1-30), maxNodes (10-2000).
   Pipeline: resolve outputDir -> callPlugin('convert_selection') -> processRasters -> write photo files to outputDir/assetsSubdir -> write flattened-UI bitmaps ONLY to outputDir/.figma-reference/source/ -> applyRasterRoles(spec) -> detect fidelity -> generate reference HTML/CSS into .figma-reference/ (inline in the result if html+css <= 16000 chars and mode inline) -> write .figma-reference/registry.json {created, banned:[flattened assetNames and file names], texts} -> build brief -> attach images.
   Result text JSON: target{framework,styling,language,outputDir}, fidelity, brief, spec, rasters{photoFiles,tracedToSvg,flattenedUi,imagesAttached,imagesOmitted,note}, assetsWritten{dir,files,failed,skipped,complete}, reference, referenceFiles{html,css,notes,fonts}, next, bridge. Then append MCP content blocks for up to 4 flattened-UI images (a text label block then an `image` block with base64 and mimeType); omit any larger than 2.5 MB base64 and report the omission.
2. **verify_conversion**(projectDir absolute): returns verify() result.

3. **get_selection**, **get_node**, **get_svg**, **get_page_outline**, **get_tokens**: thin wrappers over the plugin handlers, results wrapped with bridge meta.
4. **get_screenshot**: returns a text block (id,name,width,height) plus an image block. Description: comparison only, never use in UI.
5. **export_assets**(outputDir, id?, svgFiles?): photos only. Returns written, tracedSvg (approximate), flattenedUiNotWritten (with action "rebuild"), failed, skipped, complete.
6. **bridge_status**: pluginConnected, bridgeDir, integrity.

4.4 Raster processing and spec rewriting

processRasters: for every exported file: SVG files kept aside; otherwise classify(buf) (analyze.js). flattened-ui -> flattened list; logo-or-icon and tracing enabled -> traceSvg, on success becomes inline SVG, else photo; everything else -> photos. Classification failure falls back to role photo. applyRasterRoles rewrites the spec in place: image fills that were traced are removed and replaced by `asset:{kind:'svg', svg, reason:'traced-from-bitmap', approximate:true}`; flattened-UI fills get role 'flattened-ui' and the node gets `rasterUi:assetName` (a png asset on such a node is converted to a flattened-ui image fill and the asset removed); photos get role and `file: assetsSub/filename`. Adds spec.rasterAssets[{assetName,file,role,width,height,note}] and spec.rasterUi[{assetName,node,nodeId,width,height,palette,reason,measurements}], and updates spec.assets roles.

4.5 Fidelity detection

Override wins ("set by the user"). If any flattened UI exists -> hifi ("contains a flattened UI bitmap"). Else lofi iff grayRatio >= 0.85 AND no gradients AND distinctColors <= 8 AND effectNodes <= 2 AND (no image fills OR placeholders > 0); otherwise hifi. Return {mode, reasons[]} with human-readable reasons (e.g. "mostly neutral greys (92%)", "3 distinct colours", "no gradients", "2 image placeholder(s)").

5. Raster analysis (analyze.js) — no OCR

- decode(buf): PNG via pngjs, JPEG via jpeg-js (RGBA); GIF/WebP/other -> not analysable -> role photo, reason "format not analyzable: kept as image".
- histogram: 4096-bucket (4 bits/channel) over opaque pixels (alpha >= 250; note hasAlpha when 2 < alpha < 250). flatRatio = pixels in the 12 most common buckets / opaque pixels. sigColors = buckets with >= 0.5% share. Palette: top 6 with share %, plus sigPalette (up to 15 averaged colors).
- classify(buf) decision order: if pixels > 4,000,000 -> graphic if flatRatio >= 0.85 else photo ("too large to measure"). flat = flatRatio >= 0.8 AND sigColors <= 14; if flat and min(w,h) >= 96, run measure(); if glyphGroups >= 6 -> **flattened-ui** (reason cites the count). Else if max(w,h) <= 256, flatRatio >= 0.7, sigColors <= 10 and (png or flatRatio >= 0.9) -> **logo-or-icon**. Else flatRatio >= 0.85 and sigColors <= 14 -> **graphic**. Else **photo**. Return width, height, format, hasAlpha, flatRatio, significantColors, palette, reason, measurements (for flattened-ui).

- measure(img, hist): background = exact most-common color within the top bucket. "Card" pixels = opaque and within Manhattan RGB distance 8 of the background; card bounds = bounding box of those pixels; if the box is < 20% of the image, return background only. Corner radius: for each corner walk the 45-degree diagonal until a card pixel, d -> round(d/0.2929), cap the walk at min(60, half of the smaller side), take the median of 4. Shadow (only if hasAlpha): bounding box of semi-transparent pixels gives extent per side; approxCss from half the right-left / bottom-top difference as offsets, max extent as blur, and edge alpha (sampled 1-6 px outside the card edges, x2) as shadow opacity; flag approximate. Foreground = pixels inside the card with alpha >= 128 and distance > 48 from the background. Dilate by 2 px (separable box), 8-connected component labeling on the dilated mask, blob stats from original foreground pixels only; drop blobs < 6 pixels. Shape: circle if w,h >= 16, 0.85 <= w/h <= 1.18, fill 0.5-0.92 and all four 2x2 corners empty; line if h <= 3 and w >= 30; else glyphs. Ink color: for glyph blobs average pixels with distance >= 85% of the blob's max distance from the background (avoids anti-aliasing blends); for shapes the dominant 4-bit bucket average. Rows: cluster glyph blobs by vertical centre with tolerance max(4, 0.7*median glyph height); each row {cy, count, glyphHeight (median), colors (<=3 distinct), x: centre x of each item}. Grid: among rows with >= 4 items take the most common item count; if >= 2 rows share it, report {columns, rowsOfSameWidth, columnPitch (median x gap), rowPitch (median y gap), firstColumnCentreX, firstRowCentreY}. Output also textRows, shapes, blobCount, glyphGroups. All coordinates are in the bitmap's own pixels.
- traceSvg(buf): requires imagetracerjs. Use the image's own significant colors as the palette (plus a transparent entry if the image has transparency) so brand colors are not shifted; options ltres 1, qtres 1, pathomit 6, strokewidth 0, roundcoords 1, viewbox true, scale 1, colorquantcycles 1. Clean the output: drop paths with opacity 0, strip stroke/stroke-width/opacity=1/desc/version attributes, convert rgb() fills to hex, collapse whitespace. Reject (return null) if no paths, > 400 paths, or > 60,000 chars. Result flagged approximate.

6. Reference generator (reference.js)

Pure function generateReference(spec, {title, cssHref, assetUrl(assetName), rasterUi map}) -> {html, css, fonts[{family,weights}], notes[]}. Renders each node as a div with a generated class (f-<slug>-<counter>) and data-figma/data-component/data-hint/data-role attributes. Rules mirror the mapping rules in §7: auto layout -> flex (HORIZONTAL row / VERTICAL column; justify MIN/CENTER/MAX/SPACE_BETWEEN; align MIN/CENTER/MAX/BASELINE; gap omitted with SPACE_BETWEEN; wrap with row/column gaps; GRID -> display grid with minmax(0,1fr) columns); FILL -> flex:1 1 0 + min-size 0 on the main axis, align-self:stretch on the cross axis; HUG -> auto; FIXED -> px; absolute children -> position:absolute with left/top minus the parent's border inset; padding minus INSIDE border widths; OUTSIDE stroke -> outline; per-side borders; CENTER strokes approximated as inside borders (note it); gradient/image strokes not reproduced (note it); fills as layered backgrounds (last Figma fill on top, so reverse order); solid-only -> background-color; image fills only when a file exists (cover/contain/repeat by scaleMode); radial/conic/diamond gradients approximated (note it); ELLIPSE -> 50% radius; per-corner radii; drop/inner shadow -> box-shadow (inset for inner), text -> text-shadow, assets -> filter drop-shadow(blur/2); layer-blur -> filter blur(radius/2); background-blur -> backdrop-filter blur(radius/2);

opacity; mix-blend-mode for the supported list; rotation -> transform rotate(-deg) with origin 0 0. Text: exact family/size/weight/italic/line-height/letter-spacing/color/decoration/case; align; nowrap for WIDTH_AND_HEIGHT else pre-wrap; vertical alignment via flex when fixed-size; ENDING truncation -> ellipsis or -webkit-line-clamp; segments become spans with their own classes; links as <a>. Assets: inline `asset.svg` inside a div; else <img> from assetUrl; svgRef -> comment pointing to get_svg(id); missing -> comment + note. Placeholders: CSS box with two diagonal gradient lines (X-box) or a plain muted circle for avatar/ellipse. Flattened-UI nodes: no background image; emit a measured skeleton card (position/size/radius/shadow scaled by node size / bitmap size), accent shapes (circles/lines) as divs, and HTML comments describing each text row (count, centre y, glyph height, colors, centre x) plus a note that text must be read from the attached image. Truncated nodes get a comment with the get_node id. Output includes a Google Fonts link for the families found, `box-sizing:border-box`, and the CSS file.

7. Conversion brief (targets.js)

buildBrief({framework, styling, language, notes, outputDir, baseName, fidelity, rasters}) -> {framework, styling, language, brief}. Framework aliases (case-insensitive, spaces -> '-'): angular|ng; react|reactjs; next|nextjs; vue|vue3|nuxt; svelte|sveltekit; html|css|vanilla; flutter|dart; swiftui|swift|ios; compose|jetpack-compose|kotlin|android; react-native|rn|expo. Styling aliases: css, scss|sass, tailwind, css-modules, styled-components|emotion, inline. Defaults: styling css for web, language typescript for web frameworks except html. Unknown frameworks get a custom-target brief with generic mapping and the user's words.
Brief sections, in order: header (target, styling, language), fidelity line with reasons, output directory rule, suggested file layout per framework (Angular standalone component .ts/.html/stylesheet in kebab-case names; React/Next .tsx + style file in PascalCase; Vue .vue; Svelte .svelte; HTML index.html + styles; Flutter lib/widgets/*.dart; SwiftUI Views/*.swift; Compose ui/*.kt; React Native .tsx), PURE-CODE POLICY (svg inline counts, per-framework SVG embedding rule: JSX camelCase attributes for React/Next, SvgPicture.string for Flutter, SvgXml for React Native, Path/Canvas/ImageVector for SwiftUI/Compose; svgRef -> get_svg; only real photos as files; never screenshots, flattened bitmaps or .figma-reference content; finish with verify_conversion), REBUILD FLATTENED UI steps (read attached image text exactly; use measurements; repeating structures become real grids/lists; glyph icons redrawn as tiny SVG; font size ≈ glyphHeight/0.72; font family not measurable so choose and disclose; state what was read vs assumed), LO-FI or HI-FI rules (lo-fi: exact structure/spacing, no invented styling, neutral palette as CSS variables, image-placeholder as sized muted boxes, semantic elements from hint, accessible labels; hi-fi: pixel-exact values, hover/focus only if shown), mapping rules (web mapping list mirroring §6; per-framework rules: Angular standalone + @if/@for + signal inputs + :host display; React typed function components; Next server components by default; Vue <script setup lang="ts">; Svelte $props(); Flutter Row/Column/Expanded/Stack/Positioned/BoxDecoration/TextStyle height = lineHeight/fontSize; SwiftUI HStack/VStack/ZStack/.frame/.offset/.tracking; Compose Row/Column/Box/Modifier; React Native View/StyleSheet with explicit flexDirection, iOS shadow props vs Android elevation), styling rules (scss: variables for every token; tailwind: exact arbitrary values, never round to scale; css-modules; styled-components with theme; inline), fidelity rules (exact values, load all fonts, use tokens/vars,

repeated components -> one component with props, resolve truncation/errors via get_node, include user notes), guardrails (bridge folder is read-only; report problems in chat).

8. Verifier (verify.js)

verify(projectDir) -> {pass, violations[], warnings[], coverage, scanned{codeFiles,imageFiles,imagesReferenced}, registry:boolean}. Walk the project (max 20,000 files), skipping node_modules, .git, dist, build, out, .angular, .next, .nuxt, .svelte-kit, coverage, .dart_tool, Pods, .figma-reference, .gradle. Code extensions: html, htm, css, scss, sass, less, ts, tsx, js, jsx, mjs, vue, svelte, dart, swift, kt, kts, xml (skip files > 2 MB). Image extensions: png, jpg, jpeg, gif, webp. Violations: (1) any code mentioning `.figma-reference`; (2) code containing a banned name from registry.json; (3) a referenced image file (quoted/parenthesized path matched by regex, resolved by basename) that classify() labels flattened-ui; (4) a data:image/png|jpeg base64 URI (>= 2000 chars) that decodes to a flattened-ui; (5) design-text coverage: using registry.texts (normalized: lowercase, HTML entities decoded, whitespace collapsed, deduped) concatenated generated code must contain >= 60% of strings, else a missing-design-text violation (list up to 20 missing). Warnings: referenced flat logo/icon/graphic bitmaps ("inline SVG preferred"), large embedded base64 rasters, and fewer-than-all-text coverage. CLI: `node verify.js <dir>` prints JSON, exit 1 on failure.

9. Docs to write

- README: pure-code policy table; how to run the bridge outside any VS Code workspace and why; seal/unseal; setup steps (npm install; seal; import plugin manifest in Figma desktop; VS Code user-level MCP config `{"servers":{"design-code-bridge":{"type":"http","url":"http://localhost:3845/mcp"}}}`; copy the instructions file into the project's .github/copilot-instructions.md; gitignore .figma-reference/); per-session steps; FIGMA_BRIDGE_ALLOWED_ROOTS usage; hi-fi vs lo-fi; flattened UI flow and its dependency on a vision-capable model receiving MCP images; tools table; fidelity habits (one section at a time; never accept spec if stats.complete false); troubleshooting (/mcp GET gives 405, use /health; pluginConnected; empty mcp.json registers nothing; integrity MODIFIED); honest limits (radial/angular/diamond gradients, center/gradient strokes, complex blend modes, traced logos and measured shadows approximate, photos/illustrations stay images, GIF/WebP unanalysable, port change requires manifest and UI edits, org Copilot MCP policy).
- copilot-instructions.example.md: "Figma to code rules" with sections "Never touch the bridge" and "Pure code, never pictures" and 10 numbered steps: call convert_selection with exact framework/styling/language/outputDir/notes; everything code, inline SVG, svgRef -> get_svg; only real photos as files; rasterUi means rebuild from the attached image and disclose reading/font; lo-fi rules; use reference as numeric truth but never link to it; repeated components -> one component; resolve incomplete specs before coding and work section by section; check assetsWritten; finish with verify_conversion then compare with get_screenshot and list remaining visual differences honestly.
- DESIGN.md: your decisions where this spec was silent.

10. Tests and acceptance

Write automated tests (node:test) for: path guard (inside bridge, symlink into bridge, relative path, allowed-roots), seal/verify/unseal round trip and MODIFIED detection, no-overwrite asset writer (identical vs different content), fidelity detection thresholds, classify() on synthetic PNGs (flat calendar-like card with text-like blobs -> flattened-ui; small flat logo -> logo-or-icon; noisy gradient -> photo), measure() on a synthetic card with known radius, reference generator snapshot (auto layout, absolute, gradient, placeholder, asset svg), buildBrief for each framework alias, verify() fixtures (flattened image referenced, reference folder dependency, low text coverage, clean project passes), and an MCP smoke test (start server, POST initialize + tools/list over /mcp, assert all nine tools exist, /health shape, 405 on GET /mcp, 403 on a foreign Origin). Provide a fake-plugin WebSocket client for end-to-end tests of convert_selection.
Definition of done: `npm install && npm test` passes; `node server.js` serves /health; tools/list returns convert_selection, verify_conversion, get_selection, get_node, get_svg, get_page_outline, get_tokens, get_screenshot, export_assets, bridge_status (ten tools); no tool writes inside the bridge folder; no generated output ever contains a flattened-UI bitmap.

Start by writing DESIGN.md (decisions and the exact spec JSON shape), then implement module by module (guard, analyze, verify, reference, targets, code.js, index.html, server.js), then tests, then docs. Report anything you could not verify (for example Figma API field names) instead of guessing silently.

PART 2: ARCHITECTURAL PATTERNS (each with the invariant it must protect)

1. **Two-process bridge**: the Figma plugin sandbox (code.js) and UI iframe (index.html) talk to a local Node server over WebSocket; the server talks to the AI agent over MCP. Invariant: the plugin sandbox has no network or filesystem access, so all I/O goes through the UI and server.
2. **Request/response over a socket**: `callPlugin` sends a request with a unique ID and waits for the matching reply. Invariant: every request has an ID, a timeout, and a clean failure when no plugin is connected. Only one plugin connection is active; a new one replaces the old.
3. **Dispatch table**: `handlers[msg.tool]` in the plugin and one registered tool per capability on the server. Invariant: adding a tool means one handler plus one registration, nothing else.
4. **Pipeline with an AI-free front half**: extract spec -> analyze rasters -> generate reference HTML/CSS -> build brief -> AI writes code -> verify. Invariant: everything before the AI step is deterministic and works offline with no AI access.
5. **Decision tree for image vs code**: `decideAsset` classifies each node as code, SVG or raster. Invariant: raster is the fallback, never the default.
6. **Bounded serialization with an honesty flag**: `ser` and `newCtx` cap depth (default 12) and node count (default 600) and record what was cut. Invariant: output reports `complete: false` and the IDs of truncated children instead of silently dropping them.

7. **Strategy per target**: `buildBrief` varies instructions by framework and styling from one framework-neutral spec. Invariant: the spec never changes per framework; only the brief does.
8. **Defense in depth on the server**: Host-header check (`hostOk`), allowed output dirs, containment checks (`isWithin`), symlink-safe real-path checks (`nearestRealpath`), safe names (`safeName`). Invariant: nothing writes outside an approved folder and nothing outside localhost can call the server.
9. **Integrity sealing**: guard.js hashes the bridge files with SHA-256, writes a seal file, makes files read-only, and reports changes. Invariant: bridge files cannot change without an explicit unseal.
10. **Closed-loop verification**: `verify` checks generated output against the rules. Invariant: the AI is told to fix only what verify flags, never regenerate everything.
11. **Graceful degradation by fidelity**: `detectFidelity` and `applyRasterRoles` handle wireframes vs finished designs vs flattened bitmaps. Invariant: a flat bitmap produces a lower-fidelity path with an explicit warning.
12. **Optional capability**: imagetracerjs is loaded in try/catch. Invariant: the server starts and works fully without it.

PART 3: DEPENDENCIES, PACKAGE.JSON AND MODULE WIRING

ROLE

You are building the Node side of "figma-code-bridge" (v3.0.0). Use ONLY the dependencies and Node built-ins listed below, and use each one exactly as described. Do not add libraries. Do not replace a dependency with a different one.

1. package.json (exact requirements)

- name: `figma-code-bridge`, version: `3.0.0`
- description: "Local bridge that gives GitHub Copilot (or any MCP client) an exact, structured spec of the Figma selection and converts hi-fi and lo-fi designs to pure code in the requested stack."
- main: `server.js`
- bin: `figma-code-bridge` -> `server.js`, `figma-code-verify` -> `verify.js`
- scripts:
  - `start` = `node server.js`
  - `seal` = `node guard.js seal`
  - `verify` = `node guard.js verify`
  - `unseal` = `node guard.js unseal`
  - `check-output` = `node verify.js`

- engines: node >= 18
- dependencies (exact ranges):
  - `@modelcontextprotocol/sdk` ^1.17.0
  - `imagetracerjs` ^1.2.6
  - `jpeg-js` ^0.4.4
  - `pngjs` ^7.0.0
  - `ws` ^8.18.0
  - `zod` ^3.25.0
- No devDependencies. The module system is CommonJS (`require`), not ESM.

2. Third-party dependencies: what each is for and where it is used

2.1 `@modelcontextprotocol/sdk` (used only in server.js)

- Imports:
  - `McpServer` from `@modelcontextprotocol/sdk/server/mcp.js`
  - `StreamableHTTPServerTransport` from `@modelcontextprotocol/sdk/server/streamableHttp.js`
- Used inside `createMcpServer()`:
  - Create `new McpServer({ name: 'design-code-bridge', version: '3.0.0' })`.
  - Register ten tools with `registerTool(name, config, handler)`: `convert_selection`, `verify_conversion`, `get_selection`, `get_node`, `get_svg`, `get_page_outline`, `get_tokens`, `get_screenshot`, `export_assets`, `bridge_status`.
  - Each tool's `inputSchema` is a plain object of zod fields (see 2.5). Each handler returns the result through the shared `json(obj)` helper.
- Transport: in the HTTP request handler, create a NEW `StreamableHTTPServerTransport({ sessionIdGenerator: undefined })` per MCP request (stateless mode), connect a fresh MCP server instance to it, and let it handle the request. Do not keep sessions.
- Protocol errors are returned through the `rpcError` helper in JSON-RPC shape.

2.2 `ws` (used only in server.js)

- Import `WebSocketServer`.
- Create `new WebSocketServer({ noServer: true, maxPayload: 200 * 1024 * 1024 })`. The 200 MB cap exists because the plugin sends large base64 image exports.
- `noServer: true` means the WebSocket is attached manually: the shared `http.createServer` listens for the `upgrade` event, runs the host check, then calls `wss.handleUpgrade(req, socket, head, (ws) => attachPlugin(ws))`.

- `attachPlugin(ws)` keeps exactly one active plugin connection and replaces the older one when a new plugin connects. `shutdown()` calls `terminate()` on every client in `wss.clients`.
- Browser side: index.html uses the browser's built-in `WebSocket` (not `ws`) to connect to the local server.

2.3 `zod` (used only in server.js)

- Used to define MCP tool input schemas: `z.string()`, `z.number().int().min().max()`, `z.boolean()`, `z.enum([...])`, each with `.optional()` and `.describe()` where needed.
- Shared schemas defined once and reused:
  - `depthSchema`: integer 1-30, optional, default 12.
  - `maxNodesSchema`: integer 10-2000, optional, default 600.
- `convert_selection` fields: `framework`, `outputDir`, `styling`, `language`, `notes`, `fidelity` (enum `auto|lofi|hifi`), `name`, `id`, `assetsSubdir`, `exportAssets`, `traceRasterLogos`, `svgFiles`, `reference` (enum `inline|file|none`), plus depth and maxNodes.
- The `.describe()` text is part of the product: it is what the AI agent reads to decide how to call each tool. Write them as precise instructions.

2.4 `pngjs` (used only in analyze.js)

- Import `{ PNG }`.
- `decode(buf)` uses `PNG.sync.read(buf)` to turn a PNG buffer into raw RGBA pixel data (width, height, data).

2.5 `jpeg-js` (used only in analyze.js)

- Import as `jpeg`.
- `decode(buf)` uses `jpeg.decode(buf, { useTArray: true, formatAsRGBA: true, maxMemoryUsageInMB: 512 })` for JPEG input, so both formats yield the same RGBA structure. The memory cap protects against huge images.
- `decode` chooses PNG or JPEG by sniffing the buffer's magic bytes.

2.6 `imagetracerjs` (used only in analyze.js; OPTIONAL)

- Loaded in a `try { require('imagetracerjs') } catch` block into a variable that may stay `null`.
- `traceSvg(buf)` returns `null` immediately if the library is missing. Otherwise it decodes the bitmap and calls `ImageTracer.imagedataToSVG(...)` to turn a small flat logo or icon bitmap into SVG code.
- The result is approximate, and the server marks it as such. The tool still works fully without this package.

3. Node built-ins and where they are used

Built-in	File	Use
`fs`	guard.js, verify.js	Synchronous reads, writes, chmod, existence checks, realpath
`fs/promises`	server.js	Async directory creation and file writing for assets and output
`path`	guard.js, server.js, verify.js	Path building, `isWithin` containment checks
`crypto`	guard.js, server.js	guard.js: SHA-256 file hashes for the seal. server.js: `crypto.randomUUID()` request IDs in `callPlugin`
`http`	server.js	`http.createServer` for MCP endpoint, health check and the WebSocket upgrade

4. Local module dependency graph (CommonJS)

- `server.js` requires: `./guard`, `./targets` (`buildBrief`, `kebab`), `./reference` (`generateReference`), `./analyze` (as `A`), `./verify` (`verify`).
- `verify.js` requires: `fs`, `path`, `./analyze` (as `A`).
- `analyze.js` requires: `pngjs`, `jpeg-js`, `imagetracerjs` (optional).
- `guard.js`, `targets.js`, `reference.js`: no local requires. targets.js and reference.js are pure functions with no I/O.
- Plugin side (`code.js`, `index.html`): no npm packages at all. Runs inside the Figma sandbox using only the `figma` global and `postMessage`.
- There must be no circular requires.

5. How each function uses the dependencies (call flow)

1. **Startup (server.js)**: `guard.check()` reports seal status, then `http.createServer` starts and a `WebSocketServer` is created.
2. **Plugin connects**: HTTP `upgrade` -> `hostOk` -> `wss.handleUpgrade` -> `attachPlugin`.
3. **AI calls a tool over MCP**: `StreamableHTTPServerTransport` -> `McpServer` -> zod validates arguments -> tool handler.
4. **Handler needs Figma data**: `callPlugin(tool, args)` makes a `crypto.randomUUID()` ID, sends it over the WebSocket, and waits for the matching response or a timeout. In the plugin UI, index.html forwards it with `parent.postMessage`, code.js runs `handlers[msg.tool]`, and the reply goes back via `figma.ui.postMessage`.
5. **Output folder safety**: `resolveOutputDir` -> `ensureAllowedOutputDir` -> `guard.assertSafeOutputDir` (checked twice, including the nearest real path to defeat symlinks) -> `fs/promises` writes.
6. **Image analysis**: `processRasters` -> `A.classify(buf)` (pngjs / jpeg-js decode, then pixel analysis). If it is a small flat logo and tracing is enabled, `A.traceSvg(buf)` runs (imagetracerjs).

7. **AI-free generation**: `generateReference(spec, options)` then `buildBrief({...})`. Both are pure and need no packages.
8. **Verification**: `verify(projectDir)` walks the generated project with `fs` and re-runs `A.classify` on embedded or file images to catch flattened UI.

6. Rules for the model building this

- Install exactly the six packages in section 1 with the stated ranges.
- Never import a package that is not listed. Use built-ins where listed.
- imagetracerjs must stay optional. All other packages are required.
- Keep targets.js and reference.js dependency-free and I/O-free.
- Do not use ESM syntax, `node-fetch`, `express`, `sharp`, `canvas`, or an OCR library.
- After writing, run `npm install` then `npm start` and confirm that the server starts, `bridge_status` responds, and removing imagetracerjs does not crash startup.

Entry-point detail

- server.js and verify.js both start with the `#!/usr/bin/env node` shebang line, because package.json exposes them as `bin` commands.

PART 4: REQUIRED FUNCTION INVENTORY (names must match; behavior is defined in Part 1)

- **guard.js**: `sha`, `chmodAll`, `seal`, `unseal`, `check`, `assertSafeOutputDir`, `assertSafeSubdir`. Exports also include `BRIDGE_DIR`. Runs as a CLI for `seal`, `verify`, `unseal`.
- **targets.js** (pure): `norm`, `kebab`, `pascal`, `buildBrief`. Exports `buildBrief` and `kebab`.
- **reference.js** (pure): `esc`, `slug`, `px`, `generateReference`, `gradientLayer`, `backgroundDecls`, `typo`, `emit`, `render`, `axis`.
- **analyze.js**: `hex`, `decode`, `histogram`, `modeColor`, `dist`, `dilate`, `median`, `cornerRadius`, `measure`, `classify`, `toHexRgb`, `traceSvg`. Exports `classify` and `traceSvg`.
- **verify.js**: `walk`, `lineOf`, `norm`, `verify`. Works as a library (`verify(projectDir)`) and as a CLI.
- **code.js** (plugin main thread):
  - helpers: `r2`, `isMixed`, `has`, `toHex`, `h`, `toColor`, `saturation`, `slug`, `nodeAssetName`, `imageAssetName`, `lineHeight`, `letterSpacing`, `invert`, `relPos`
  - variables and styles: `varName`, `styleName`
  - paint and style: `paint`, `paints`, `effects`, `radius`, `imageFillHashes`
  - analysis: `stats`, `onlyShapes`, `hasMaskChild`, `decideAsset`, `isXBox`, `detectRole`
  - core: `ser`, `newCtx`, `buildSpec`

  - export: `sniffImage`, `exportAll`
  - selection: `formatValue`, `resolveNode`, `resolveRoots`, `announceSelection`
  - `handlers` object: `get_selection`, `get_node`, `convert_selection`, `export_assets`, `get_svg`, `get_page_outline`, `get_screenshot`, `get_tokens`
- server.js: `callPlugin`, `attachPlugin`, `meta`, `json`, `safeName`, `isWithin`, `nearestRealpath`, `ensureAllowedOutputDir`, `resolveOutputDir`, `writeAssets`, `walkNodes`, `processRasters`, `detectFidelity`, `applyRasterRoles`, `createMcpServer`, `hostOk`, `readBody`, `rpcError`, `shutdown`. `createMcpServer` registers: `convert_selection`, `verify_conversion`, `get_selection`, `get_node`, `get_svg`, `get_page_outline`, `get_tokens`, `get_screenshot`, `export_assets`, `bridge_status`.
- index.html: `setConnectionState`, `updateLiveState`, `showSelectionMessage`, `addActivity`, `showSelection`, `connect` (with WebSocket open, message, close and error handlers; forwards requests to the plugin with `parent.postMessage` and relays responses back).

PART 5: REQUIREMENTS FOR COMPLEX DESIGNS

The tool must work on large, grouped, vector-heavy, partly flattened designs, not only simple screens.

1. **Preflight audit**: before converting, scan the selection and report vectorized text, flattened bitmaps, unnamed layers, missing auto-layout and detached components. Return a convertibility score and a list of problem node IDs, so failures are explained before any conversion starts.
2. **Completeness guarantee**: every visible text node, logo, nav item and button label in the spec must appear in the generated code. `verify` must fail when any is missing.
3. **Chunked conversion**: convert one top-level section at a time (nav, hero, footer, and so on) and compose afterwards. When `ser` truncates, return the IDs of cut children so the agent fetches them with `get_node` instead of guessing.
4. **Layout inference**: for frames without auto-layout, cluster children by position into rows and columns. Prefer flex or grid. Use absolute positioning only for true overlaps.
5. **Vector and logo policy**: logos and icons become inline SVG via `get_svg`, and are never dropped. Small flat logo bitmaps are traced (approximate, flagged). Photos become raster files. Text is never an image.
6. **Text robustness**: use min-height, not fixed height, so extra lines never clip. Handle mixed-style runs, missing fonts (fallback stack), letter-spacing and line-height.
7. **Components and tokens**: repeated instances become one component with props. Figma variables and styles become CSS variables or theme tokens.
8. **Visual diff loop**: render the generated code, screenshot it, compare with the Figma export and fix only the regions that differ.
9. **Edge cases handled explicitly**: masks, blend modes, gradients, rotated and overlapping nodes, hidden layers, off-canvas elements, constraints, responsive behavior, prototype states such as hover

variants.

10. **Honest failure reporting**: every run ends with a list of what was approximated, skipped or unresolved.
11. **Cost control** (AI usage may be limited and shared): run all AI-free steps first (spec, analysis, reference HTML/CSS, preflight). Send the AI only one section at a time, with the reference as the base. Support `reference: file | none`, `exportAssets: false`, lower depth and maxNodes. Ask the AI to fix only what verify flags.
12. **Offline operation**: all analysis, reference generation and verification must work with no AI access and no network.

PART 6: DELIVERY, TESTS AND ACCEPTANCE

1. Deliver files in this order: package.json, guard.js, targets.js, reference.js, analyze.js, verify.js, server.js, code.js, manifest.json, index.html, README.md, DESIGN.md, a Copilot instructions example file.
2. The manifest.json must declare the plugin entry (code.js), the UI (index.html), network access limited to the local server, and the Figma editor type.
3. Acceptance checks (run them and report results):
   - `npm install` then `npm start` succeeds; `bridge_status` responds.
   - The server still starts with imagetracerjs removed.
   - Requests from a non-local Host header are rejected.
   - An output directory outside the allowed area, or a symlink escape, is rejected.
   - `seal` then editing a file makes `verify` report the change.
   - Three fixed test designs (simple, medium, complex) pass: 100% text coverage, zero dropped logos, `complete: true` or an explicit truncation report.
   - No generated output contains an image in place of text.
4. Finish with a short report: what was built, what was approximated, and what could not be verified.